Results

Run snapshot. With the bundled `manuscript/config.yaml` the most recent execution evaluated the query *"reproducible research optimization"* against local, returned 6 deduplicated paper(s) (4 carrying a DOI, 6 carrying an abstract); the per-source breakdown is local=6 and recorded backend errors are none. The deep-search workflow (sec. `sec:deep_search`) covered 3 keyword(s) — *convex optimization; stochastic gradient descent; reproducible research* — drawn from , producing unique paper(s) after cross-keyword deduplication.

When `sources: [local]` is used, this section reports fixture execution only. It must not be read as a claim about empirical literature coverage; a live-provider run requires source-level provenance and a separately reviewed claim boundary.

The diagnostic figures generated for this run are catalogued in sec. `sec:methodology`: fig. **??** surfaces per-backend coverage, fig. **??** surfaces the temporal distribution, and fig. **??** surfaces the relevance-score profile. The full determinism contract for each stage is itemised in tbl. **??** of sec. `sec:reproducibility`.

Interpreting the run snapshot I

The numerical values in the run-snapshot paragraph that opens this section are read directly from `output/run_summary.json` and `output/data/manuscript_variables.json` so they always reflect the most recent run rather than a stale claim hand-typed into the prose. Three properties are worth highlighting (the formal determinism contract for each underlying stage is enumerated in tbl. **??**):

Interpreting the run snapshot I

- **Cache reuse is observable.** A second invocation against the same config produces a byte-identical artifact tree (modulo the wall-clock timestamp inside `output/search/cache/search_<hash>.json` itself); see also the *Search (cached hit)* row of tbl. **??** and the verification recipe in sec. `sec: reproducibility`. The cache file thus doubles as a cryptographic seal: re-running is a file read, not a network round-trip.
- **Deduplication is signal-preserving.** The aggregator merges by DOI $\rightarrow$ arXiv id $\rightarrow$ normalised (title, year), keeping the highest-scored copy and filling missing fields from the loser (see *Dedup / merge* in tbl. **??** and the per-backend pre-dedup view in fig. **??**). The `RESULT_NUM_PAPERS` figure therefore equals "papers a reviewer needs to read", not "raw backend hit count" — the per-source contributions in `RESULT_PER_SOURCE` are the pre-dedup view.
- **Enrichment coverage is honest.** `RESULT_WITH_ABSTRACT` and `RESULT_WITH_DOI` count fields the corpus or the `AbstractFetcher` actually populated, never values inferred. When a paper is missing a DOI it is excluded from the DOI count even if its arXiv id resolves to one upstream. The temporal coverage of those papers is summarised by fig. **??**, and their backend-reported relevance scores by fig. **??**.

Output artefacts I

After running `scripts/run_search_pipeline.py` against the default `manuscript/config.yaml`, the project produces:

Output artefacts I

- `output/search/results.json` — the raw `SearchResult` JSON, including `per_source_counts` and `errors` for diagnostic purposes.
- `output/search/cache/search_<hash>.json` — the deterministic search cache; identical reruns are file reads.
- `output/cache/abs/<safe_id>.txt` — one file per fetched abstract.
- `output/cache/pdf/<safe_id>.{pdf,txt}` — PDFs and extracted text (only when `enrichment.fetch_fulltext: true`).
- `output/corpus.json` — a `LocalBackend`-compatible JSON corpus of every result, enriched in place.
- `manuscript/references.bib` — the auto-populated bibliography from the single-query pipeline (merged with any other `manuscript/*.bib` at PDF render time).
- `output/llm/per_paper/<safe_id>.md` — per-paper LLM analyses (only when `llm.per_paper: true` *and* the LLM stack is reachable).
- `output/llm/synthesis.md` — corpus-level LLM synthesis (only when `llm.corpus_synthesis: true` *and* the LLM stack is reachable).
- `output/reading_report.md` — the final assembled reading report.

Output artefacts I

When the LLM stack is genuinely unreachable, the `output/llm/` artefacts are simply absent — no placeholder file is ever written into the archive (see sec. `sec:pipeline_internals`).

Because the search cache and abstract cache are deterministic, a second run with identical `config.yaml` produces byte-identical artifacts (modulo timestamp metadata in the cache files themselves). This is the property the project exists to demonstrate.

Output artefacts I

The exact paper count, DOI list, and synthesis text depend on the live state of arXiv and Crossref at the time of the run — and are therefore not reproducible *across* runs in different weeks. Users seeking strict reproducibility should:

Output artefacts I

1. Pin a `LocalBackend` corpus generated from a successful run (`infrastructure.search.literature.write_corpus`) and remove `arxiv` / `crossref` from `project_config.search.sources`.
2. Commit the `output/search/cache/` directory to version control.
3. Pin the LLM seed (`config.llm.seed`) and avoid model upgrades.

Output artefacts I

With those three steps, every run from the same commit produces the same outputs.